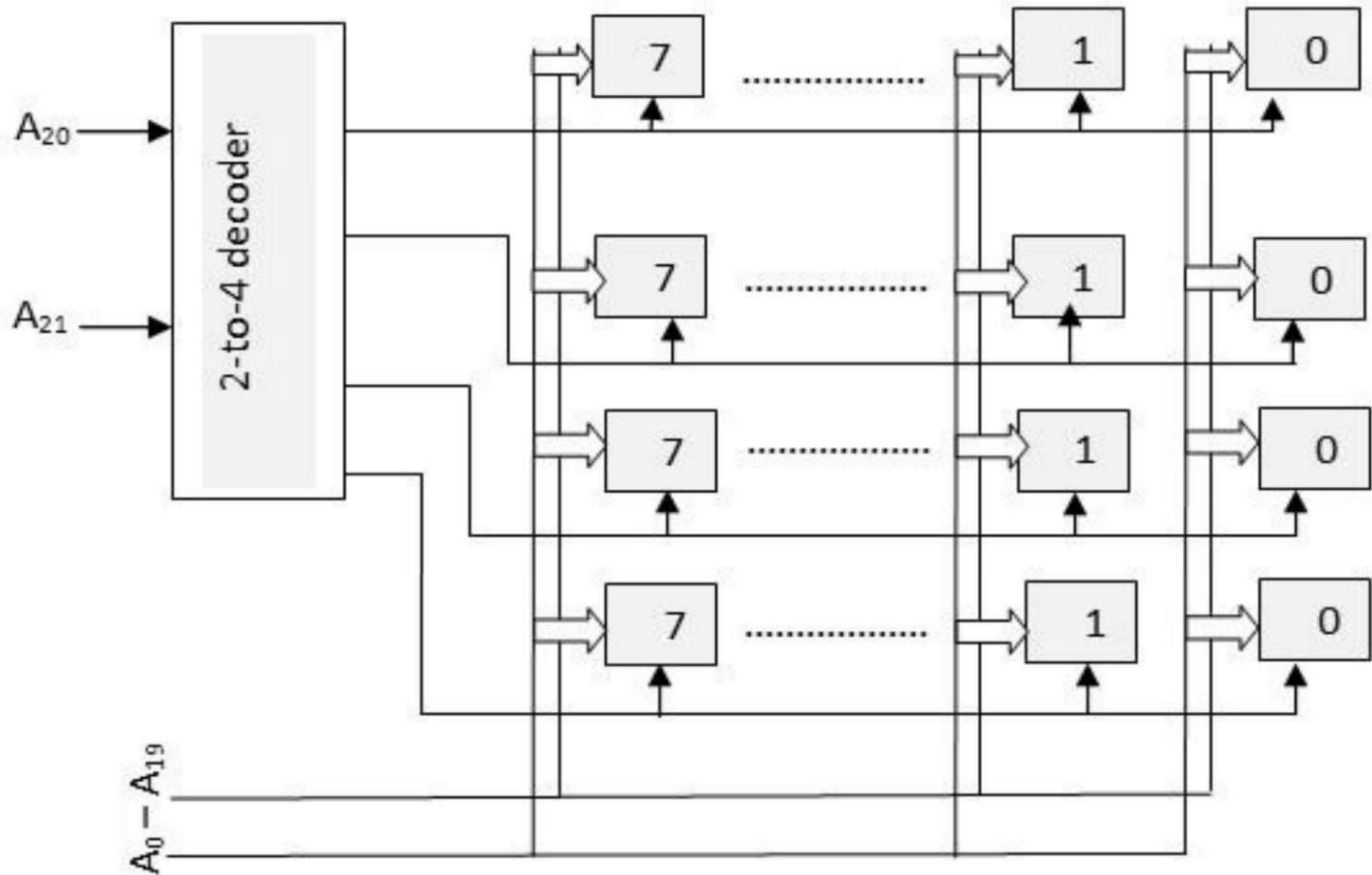
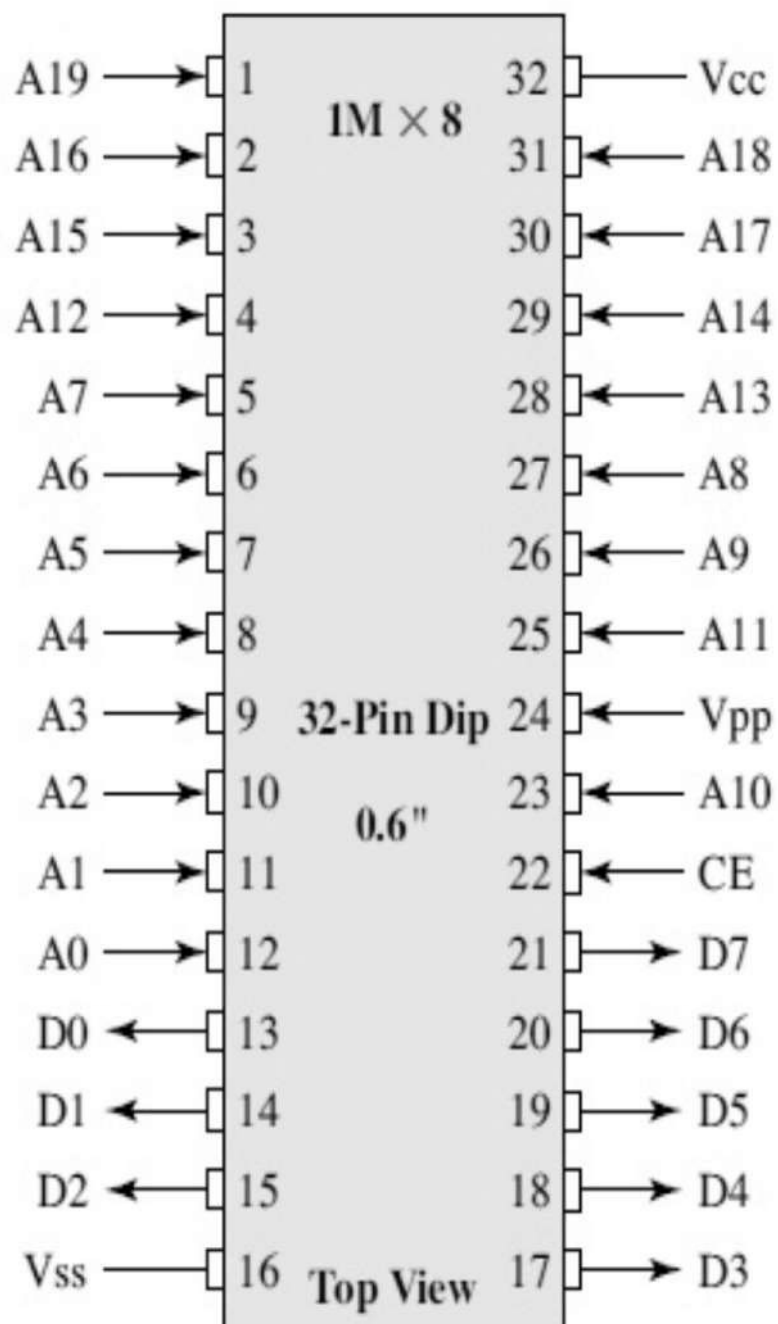
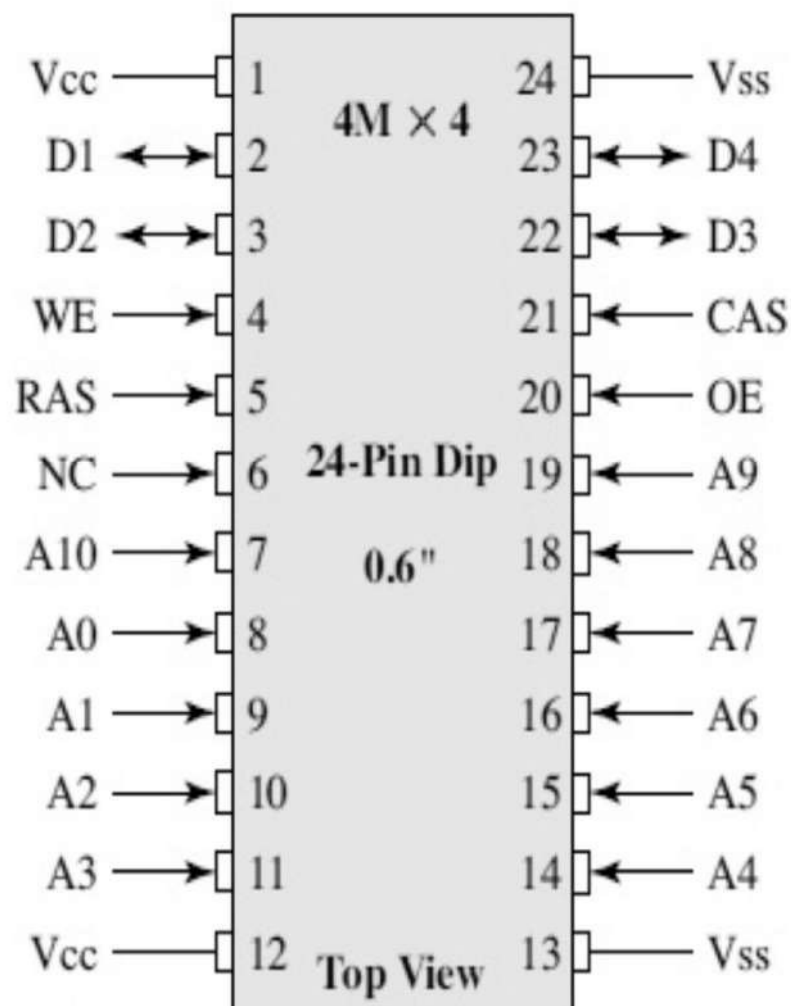


Memory Organization





(a) 8-Mbit EPROM



(b) 16-Mbit DRAM

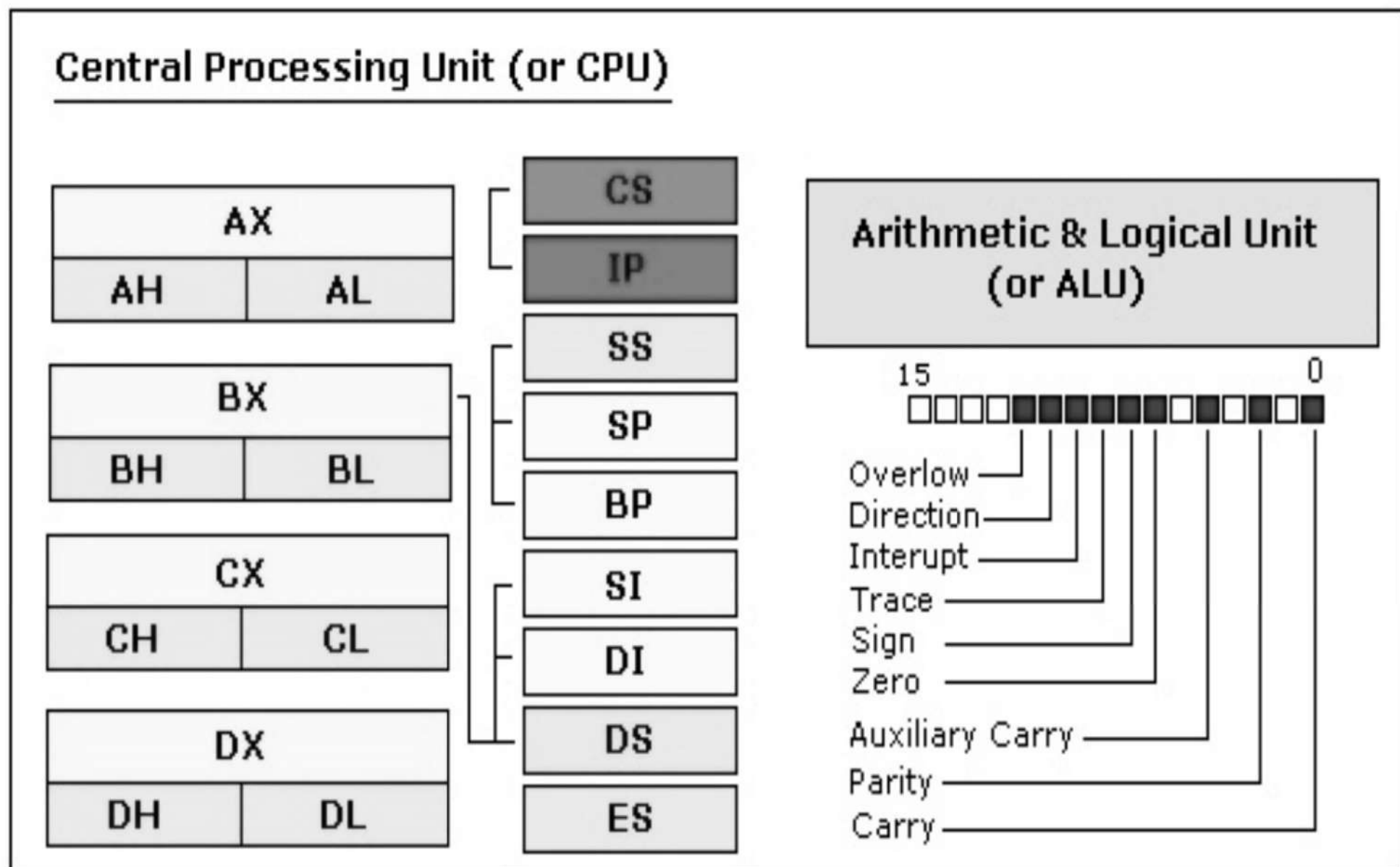
CPU Registers

- A small amount of storage available as part of a digital processor,
- Built from fast multi-ported memory cell.
- Drive its data onto an internal bus in a single clock cycle.
- Registers are measured by the number of bits they can hold
 - 16-bit register or a 32-bit register.

Registers Types

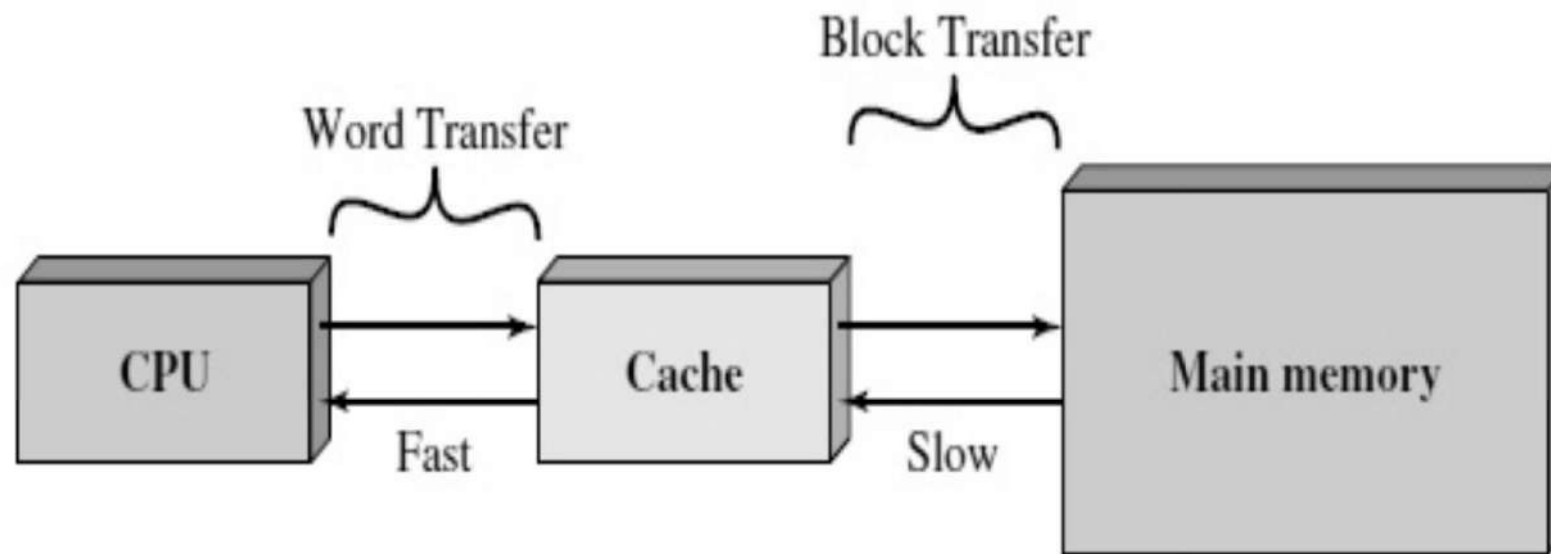
- **General Purpose Registers:** used to store data and intermediate results during program execution. Its contents can be accessed through assembly programming, as AX, BX, CX and DX registers.
- **Special Purpose Registers:** Users do not access these registers. These are used by computer system at the time of program execution as MAR, MBR, IR and PC registers.
- **Segment Registers:** as CS, DS and SS registers.
- **Special Kernel Mode Registers:** are intended for writing operating systems, debuggers, and other system level tools.

CPU Registers



Cache Memory

- a small amount of fast memory placed between two levels of memory hierarchy to bridge the gap in access times between processor and main memory and between main memory and disk.



(a) Single cache

Cache Memory

- Keep the information expected to be used more frequently by the CPU in the cache.
- When the processor makes a request for a memory reference, the request is first sought in the cache.
 - If the request corresponds to an element that is currently residing in the cache, we call that a **Cache Hit**.
 - if the request corresponds to an element that is not currently in the cache, we call that a **Cache Miss**.
- A Cache Hit Ratio (h) :the probability of finding the requested element in the cache.
- Hit Time :how long it takes data to be sent from the cache to the processor.

Cache Memory

- Cache Miss Ratio ($1 - h$) : the probability of not finding the requested element in the cache.
- Miss Penalty : the time to copy data from main memory to the cache.
- Average Memory Access Time(AMAT) can be computed as:

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Ratio} \times \text{Miss Penalty})$$

Cache Memory

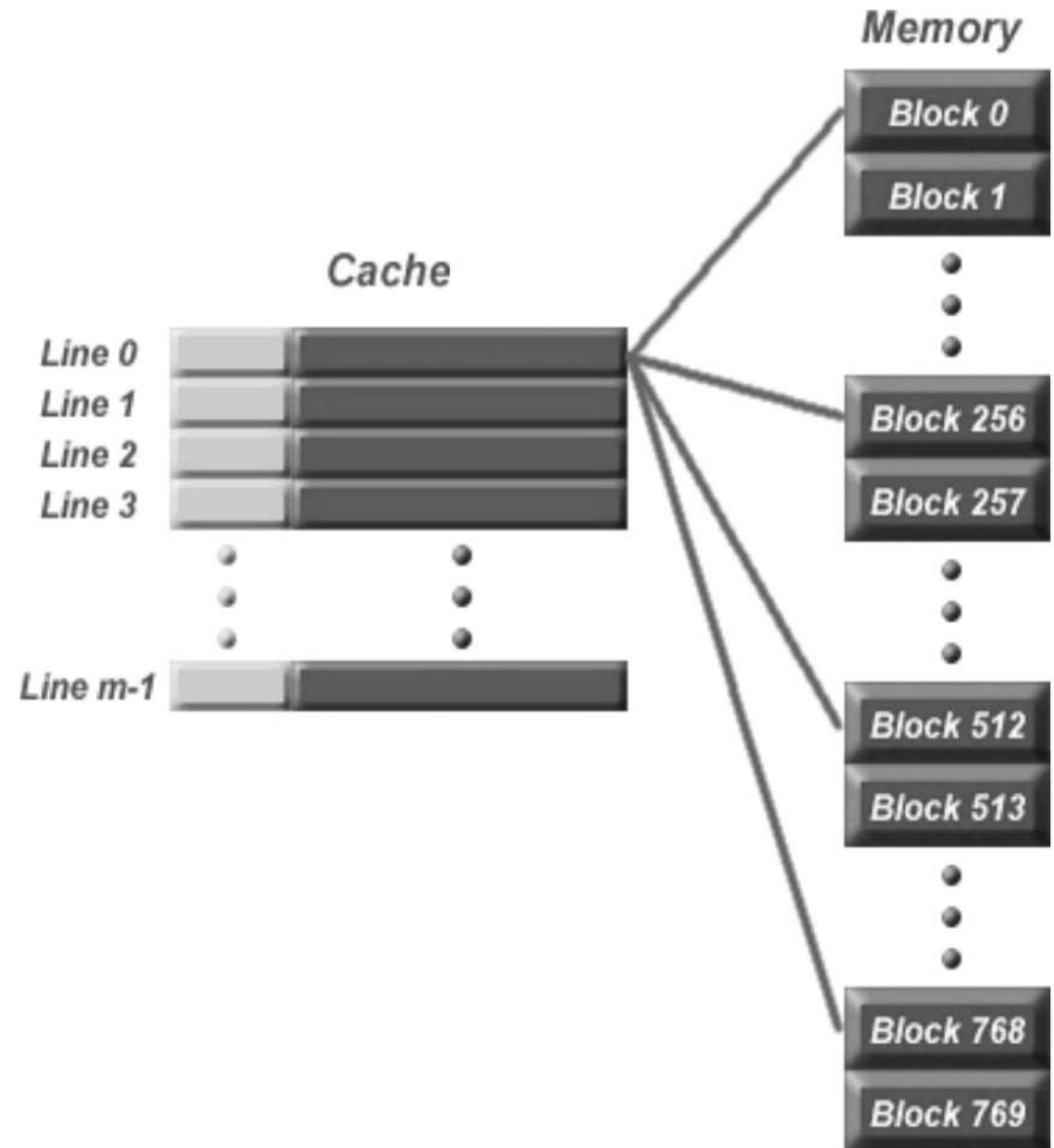
- **Assume that**
 - Hit time = 5 ns
 - Miss rate = 10%
 - Miss penalty = 100 ns.
 - The average memory access time = 15 ns.

Cache Memory Types

- Classified based on the methods of addressing to locate data within a cache into three types:
 - Direct Mapped
 - Fully Associative
 - Set Associative

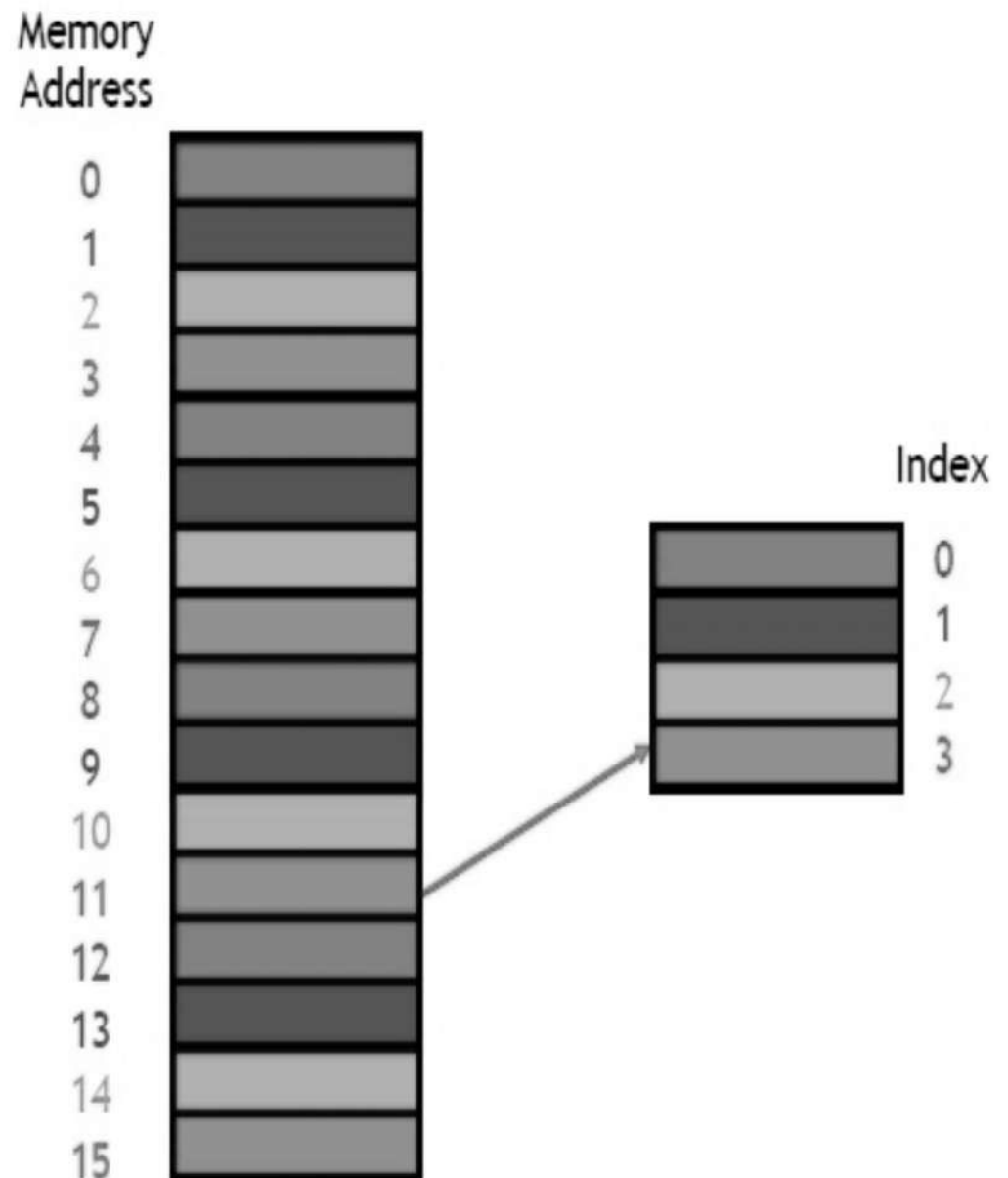
Direct Mapped Cache Memory

- Each memory block is assigned to a specific line in the cache.
- As the cache is much smaller than memory so we can find many memory blocks assigned to the same cache line.



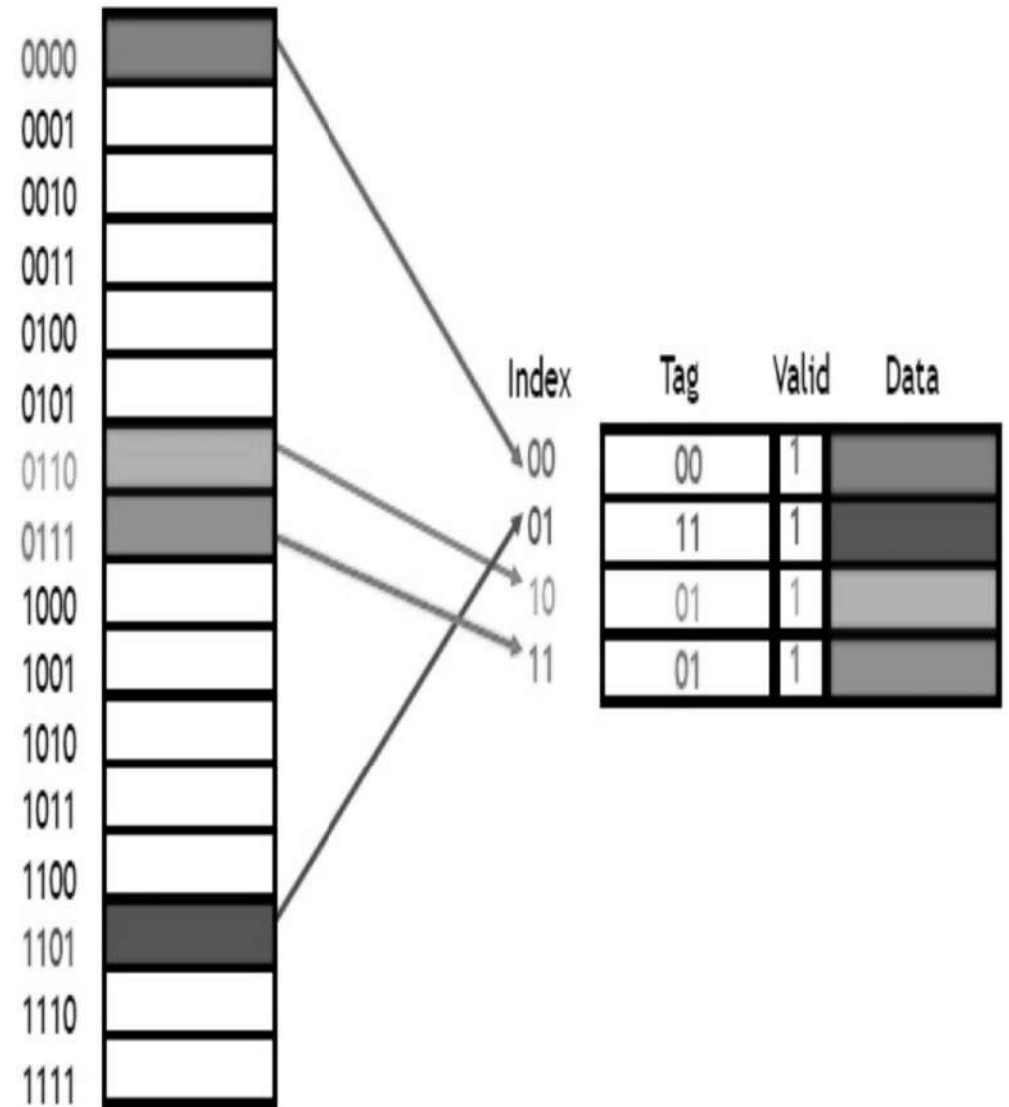
Direct Mapped Cache Memory

- If the cache contains 2^k bytes then the k least significant bits are used as index and the data from memory address i would be stored in block $i \bmod 2^k$.
- Ex: data from memory address 11 maps to cache block 3 since $11 \bmod 4 = 3$ or since the lowest two bits of 1011 are 11.

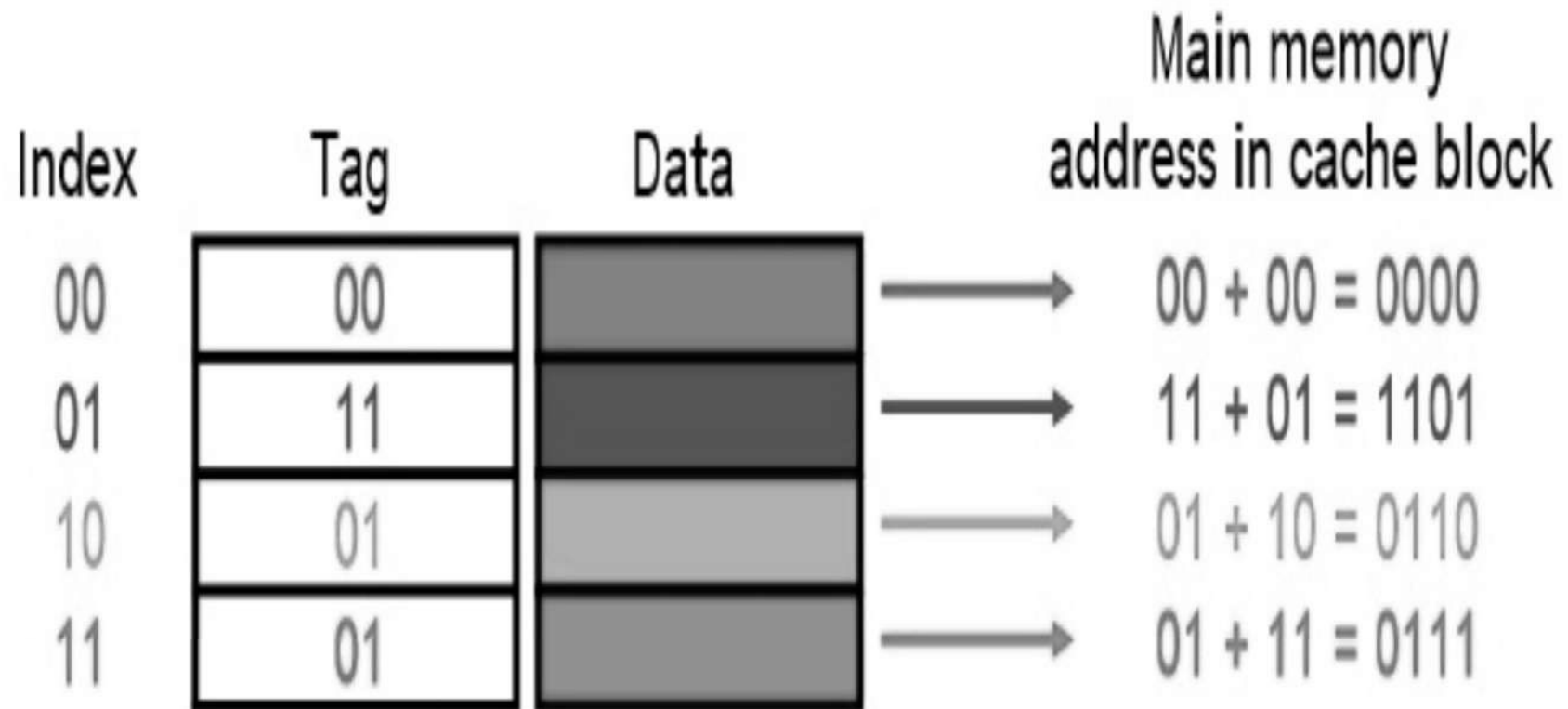


Direct Mapped Cache Memory

- To find data stored in the cache:
 - Tags are added to distinguish between different memory locations that map to the same cache block
 - A single valid bit per block is added to distinguish full and empty blocks.



Direct Mapped Cache Memory

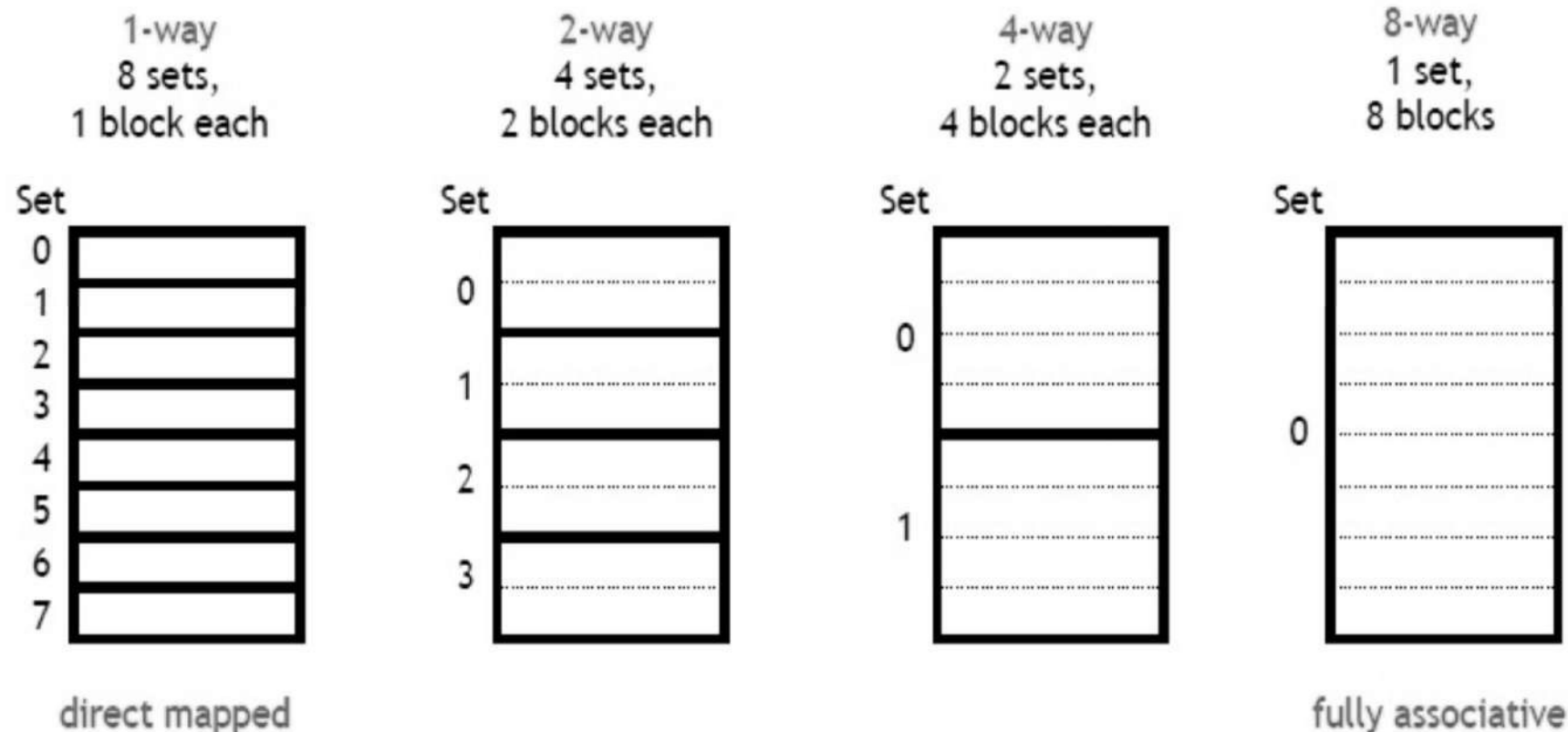


Fully Associative Cache Memory

- Permits data to be stored in any unused cache block, instead of forcing each memory address into one particular block.
- Never be a conflict between two or more memory addresses which map to a single cache block.
- If all the blocks are already in use, it's usually best to replace the least recently used one, assuming that if it hasn't been used it in a while, it won't be needed again anytime soon.
- The most expensive, due to the cost of the associative memory.
- The fully associative cache can only be formed economically with a moderate size capacity.
- Microprocessors with small internal caches often employ the fully associative mechanism.
-

Set Associative Cache Memory

- Considered as a compromise between a fully associative cache and a direct mapped cache.
- Cache is divided into groups of blocks, called sets.
- Each memory address maps to exactly one set in the cache, but data may be placed in any block within that set.
- The number of blocks in a set is known as the associativity or set size



Set Associative Cache Memory

- Each block in each set has a stored tag
- Tag with the index, completes the identification of the block.
- First, the index of the address from the processor is used to access the set. Then, comparators are used to compare all tags of the selected set with the incoming tag. If a match is found, the corresponding location is accessed.

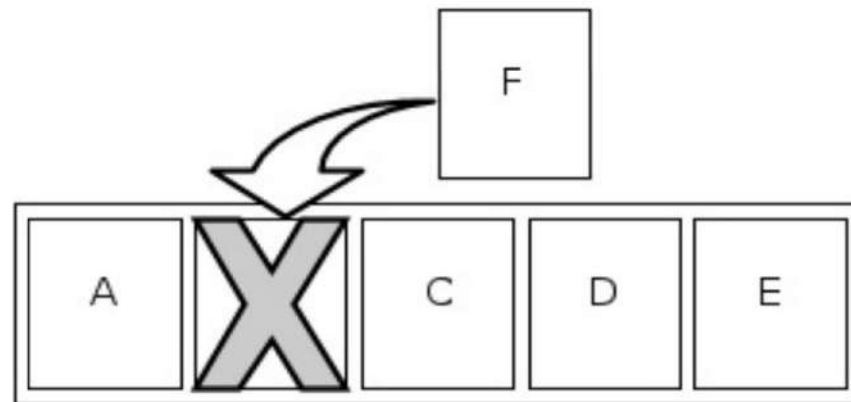
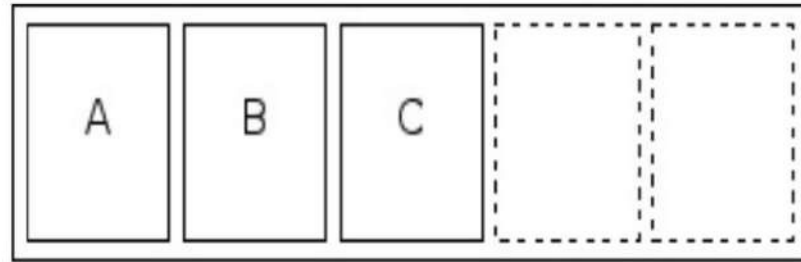
Mapping technique	Simplicity	Associative tag search	Expected cache utilization	Replacement technique
Direct	Yes	None	Low	Not needed
Associative	No	Involved	High	Yes
Set-associative	Moderate	Moderate	Moderate	Yes

Replacement Policies

- When there is no place in cache to load the memory block an effective replacement policy is needed.
 - Least Recently Used (LRU)
 - Random Replacement
 - FIFO

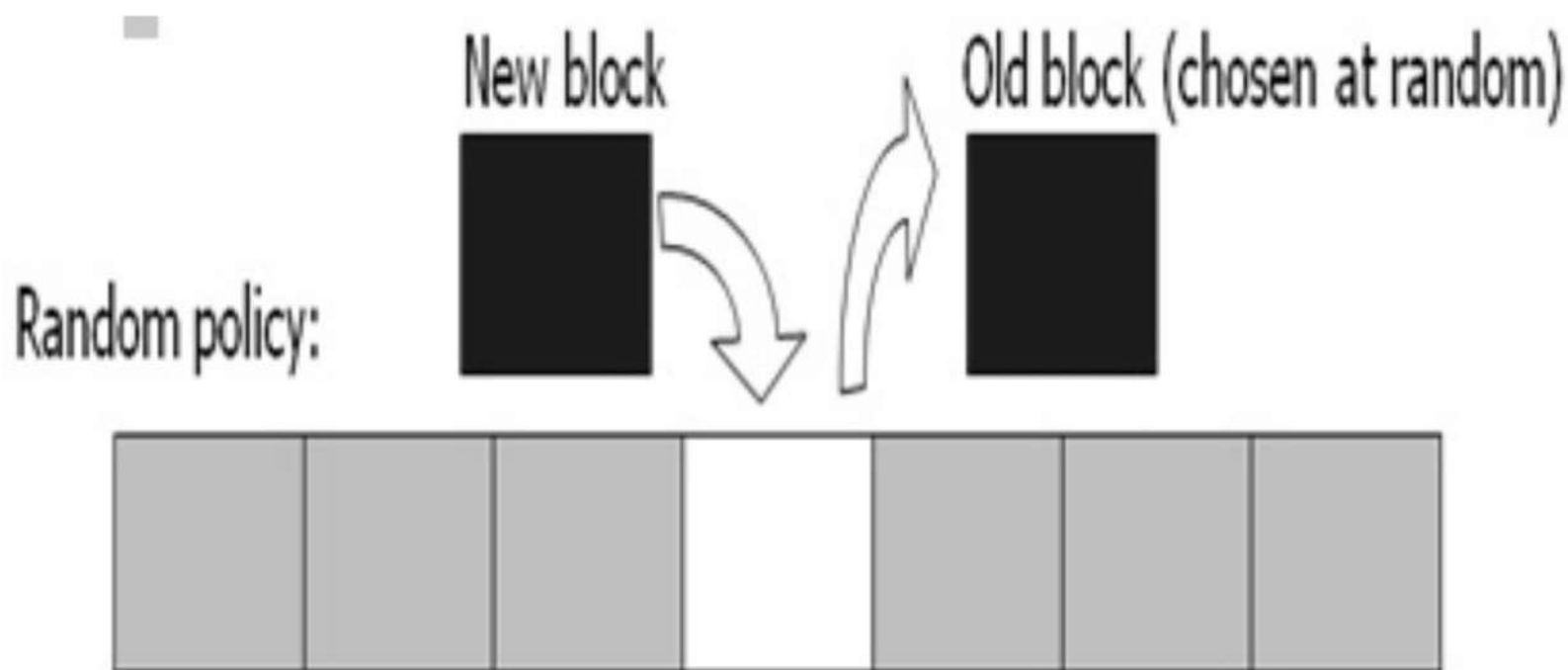
Least Recently Used (LRU)

- When performing LRU caching, you always throw out the data that was least recently used.



Random Replacement

- select a block to replace in a totally random order, with no regard to memory references or previous selections.



FIFO

- removes the block that has been in the cache for the longest time.

